

Data Exfiltration Detection

2026/02/05

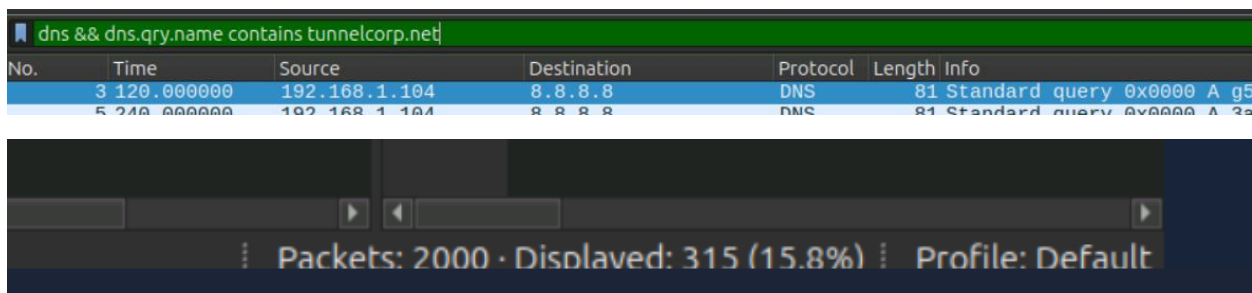
DNS exfiltration abuses the Domain Name System, a protocol normally allowed through networks, to smuggle bytes encoded inside DNS queries/responses so firewalls and web proxies don't notice. Because DNS is typically allowed and often unfiltered or forwarded to public resolvers, it's attractive for covert channels.

Step 1: What is the suspicious domain receiving the DNS traffic?

No.	Time	Source	Destination	Protocol	Length	Info
DNS	56	Standard query	0x0000	A	github.com	
DNS	59	Standard query	0x0000	A	wikipedia.org	
DNS	81	Standard query	0x0000	A	g5ft5ma3dtb1i4q7im4r.tunnelcorp.net	
DNS	56	Standard query	0x0000	A	google.com	
DNS	81	Standard query	0x0000	A	3arwfuze7kctrf0e2ef5.tunnelcorp.net	
DNS	56	Standard query	0x0000	A	github.com	
DNS	56	Standard query	0x0000	A	google.com	
DNS	59	Standard query	0x0000	A	wikipedia.org	
DNS	59	Standard query	0x0000	A	wikipedia.org	
DNS	81	Standard query	0x0000	A	7qq6jqz7qj0jdp67lesa.tunnelcorp.net	
DNS	63	Standard query	0x0000	A	stackoverflow.com	
DNS	63	Standard query	0x0000	A	stackoverflow.com	
DNS	56	Standard query	0x0000	A	github.com	
DNS	56	Standard query	0x0000	A	google.com	

Before applying any filters, I observed a number of suspicious-looking subdomains which all originated from the same root domain—**tunnelcorp.net**.

Step 2: How many suspicious traffic/logs related to dns tunneling were observed?



No.	Time	Source	Destination	Protocol	Length	Info
3	120.000000	192.168.1.104	8.8.8.8	DNS	81	Standard query 0x0000 A g5
5	240.000000	192.168.1.104	8.8.8.8	DNS	81	Standard query 0x0000 A 3a

Packets: 2000 · Displayed: 315 (15.8%) · Profile: Default

I applied the filter **dns&&dns.qry.name contains tunnelcorp.net**, which would filter out all the suspicious logs. The number of these packets – **315**.

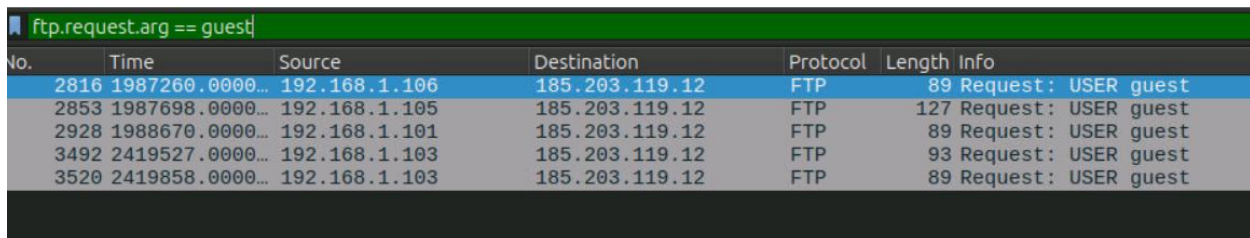
Step 3: Which local IP sent the maximum number of suspicious requests?

Ethernet		IPv4 · 5	IPv6	TCP	UDP · 315					
Address A	Address B	Packets	Bytes	Total Packets	Percent Filtered	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start
192.168.1.101	8.8.8.8	58	5 kB	406	14.29%	58	5 kB	0	0 bytes	180.000000
192.168.1.102	8.8.8.8	60	5 kB	370	16.22%	60	5 kB	0	0 bytes	480.000000
192.168.1.103	8.8.8.8	72	6 kB	405	17.78%	72	6 kB	0	0 bytes	60.000000
192.168.1.104	8.8.8.8	62	5 kB	413	15.01%	62	5 kB	0	0 bytes	120.000000
192.168.1.105	8.8.8.8	63	5 kB	406	15.52%	63	5 kB	0	0 bytes	0.000000

I opened up statistics > conversations > IPV4 and identified the IP with the maximum number of suspicious requests – **192.168.1.103**.

FTP (File Transfer Protocol) is one of the oldest protocols for transferring files between a client and server over a TCP/IP network. Attackers use it to move large amounts of data off a network, sometimes via compromised credentials, misconfigured servers, or ephemeral accounts. Detection relies on a mix of packet inspection (FTP only), server logs, SSH session metadata, and network flow/size/pattern analysis.

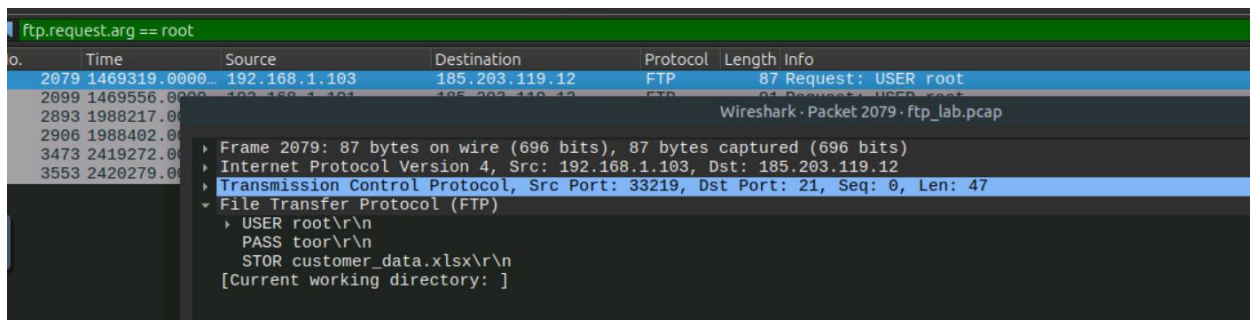
Step 1: How many connections were observed from the guest account?



No.	Time	Source	Destination	Protocol	Length	Info
2816	1987260.0000...	192.168.1.106	185.203.119.12	FTP	89	Request: USER guest
2853	1987698.0000...	192.168.1.105	185.203.119.12	FTP	127	Request: USER guest
2928	1988670.0000...	192.168.1.101	185.203.119.12	FTP	89	Request: USER guest
3492	2419527.0000...	192.168.1.103	185.203.119.12	FTP	93	Request: USER guest
3520	2419858.0000...	192.168.1.103	185.203.119.12	FTP	89	Request: USER guest

I applied the filter **ftp.request.arg==guest** to filter out all the guest account connections and identified only 5.

Step 2: Apply the filter; what is the name of the customer-related file exfiltrated from the root account?



No.	Time	Source	Destination	Protocol	Length	Info
2079	1469319.0000...	192.168.1.103	185.203.119.12	FTP	87	Request: USER root
2099	1469556.0000...	192.168.1.101	185.203.119.12	FTP	89	Request: USER root
2893	1988217.0000...	192.168.1.101	185.203.119.12	FTP	89	Request: USER root
2906	1988402.0000...	192.168.1.101	185.203.119.12	FTP	89	Request: USER root
3473	2419272.0000...	192.168.1.103	185.203.119.12	FTP	89	Request: USER root
3553	2420279.0000...	192.168.1.103	185.203.119.12	FTP	89	Request: USER root

Wireshark · Packet 2079 · ftp_lab.pcap

Frame 2079: 87 bytes on wire (696 bits), 87 bytes captured (696 bits)

Internet Protocol Version 4, Src: 192.168.1.103, Dst: 185.203.119.12

Transmission Control Protocol, Src Port: 33219, Dst Port: 21, Seq: 0, Len: 47

File Transfer Protocol (FTP)

USER root\r\n

PASS toor\r\n

STOR customer_data.xlsx\r\n

[Current working directory:]

I applied the filter **ftp.request.arg==root** and clicked on the 1st packet. There I was able to open The FTP and identified the file – **customer_data.xlsx**.

Step 3: Which internal IP was found to be sending the largest payload to an external IP?

Address A	Port A	Address B	Port B	Packets	Bytes	Stream
192.168.1.101	11474	185.203.119.12	21	1	93 bytes	
192.168.1.101	57074	185.203.119.12	21	1	91 bytes	
192.168.1.103	8825	185.203.119.12	21	1	93 bytes	
192.168.1.103	10829	185.203.119.12	21	1	93 bytes	
192.168.1.103	33732	185.203.119.12	21	1	93 bytes	
192.168.1.104	16413	185.203.119.12	21	1	92 bytes	
192.168.1.104	50742	185.203.119.12	21	1	91 bytes	
192.168.1.105	35585	185.203.119.12	21	1	127 bytes	
192.168.1.106	47729	185.203.119.12	21	1	93 bytes	
192.168.1.106	52540	185.203.119.12	21	1	93 bytes	

I applied the filter **ftp && frame.len > 90**, I then clicked Statistics > Conversations > TCP and identified the internal IP sending the largest payload – **192.168.1.105**.

Step 4: What is the flag hidden inside the ftp stream transferring the CSV file to the suspicious IP?

ftp contains "csv"

Time: 2099.1469556, 2853.1987698, 3492.2419527, 3553.2420279

File Transfer Protocol (FTP)

- USER guest\r\n
- PASS guest\r\n
- STOR internal_passwords.csv\r\n
- DATA: THM{ftp_exfil_hidden_flag}\r\n
- [Current working directory:]

0000 45 00 00 7f 00 01 00 00 40 06 87 8f c0 a8 01 69 E.....@.....i

0010 b9 cb 77 0c 8b 01 00 15 00 00 00 00 00 00 00 00 ..W.....

0020 50 02 20 00 09 b6 00 00 55 53 45 52 20 67 75 65 P.....USER gue

0030 73 74 0d 0a 50 41 53 53 20 67 75 65 73 74 0d 0a st..PASS guest..

0040 53 54 4f 52 20 69 6e 74 65 72 6e 61 6c 5f 70 61 STOR int ernal_pa

0050 73 73 77 6f 72 64 73 2e 63 73 76 0d 0a 44 41 54 sswords. csv..DAT

0060 41 3a 20 54 48 4d 7b 66 74 70 5f 65 78 66 69 6c A: THM{f tp_exfil

0070 5f 68 69 64 64 65 6e 5f 66 6c 61 67 7d 0d 0a _hidden_flag)..

Frame 2853: 11...
Internet Proto...
Transmission C...

I applied the filter **ftp contains "csv"** and looked through the packets, I managed to identify the flag – **THM{ftp_exfil_hidden_flag}** in the guest packet.

Data exfiltration via HTTP is when an attacker moves sensitive data out of a target network using HTTP as the transport. HTTP is commonly abused because it blends with normal web traffic, can traverse firewalls and proxies, and can be obfuscated (encoding, encryption, tunneling). This detection task aims to teach SOC analysts how to identify signs of HTTP-based exfiltration in packet captures (Wireshark) and logs (Splunk), and to provide practical search queries and investigation steps.

Step 1: Which internal compromised host was used to exfiltrate this sensitive data?

1 index="data_exfil" sourcetype="http_logs" method=POST bytes_sent > 600 table _time src_ip uri domain dst_ip bytes_sent sort - bytes_sent				
/ 1 event (before 2/6/26 8:11:20.000 AM) No Event Sampling ▼				
Events Patterns Statistics (1) Visualization				
Show: 20 Per Page ▼ Format Preview: On				
_time	src_ip	uri	domain	
2025-09-18 00:00:11	192.168.1.103	/v1/sync/upload	api.cloudsync-services.com	

Based on the query, I identified the compromised host by looking at the `src_ip` associated with the highest volume of outbound traffic. Since the results are sorted by `bytes_sent` in descending order, the host at the top of the list— 192.168.1.103 is the primary suspect. It sent a significant amount of data via a `POST` request to an external domain(`api.cloudsync-services.com`), which is characteristic of a data exfiltration event.

Step 2: What's the flag hidden inside the exfiltrated data?

```
tcp.stream eq 3586
o. Time Source
3589 1468805.0000... 192.168.1.103

# Internal Access Credentials - Finance Department
Username: finance_admin
Password: Fin@nc3#2025
VPN Gateway: vpn.corpnet.local
SSH Key Fingerprint: SHA256:9f:3a:2b:7c:1d:8e:4f:aa:bb:cc:dd:ee:ff:00:11:22
Database Connection:
Host: db01.internal.corp
Port: 5432
User: db_reader
Password: R3@d0nly!2025
---
Incident Response Notes (Confidential)
- Suspicious HTTP POST traffic observed from 10.10.10.104
- Payloads contain raw plaintext chunks
---
TryHackMe Flag:
THM{http_raw_3xfiltr4t10n_succ3ss}
---
File Hashes:
secret.txt: 9c8f3e2b7a1d4e5f6c7b8a9d0e1f2a3b
backup.tar.gz: 7f6e5d4c3b2a1908a7b6c5d4e3f2a1b0
# End of file
```

I applied the filter - **http.request.method == "POST" and frame.len > 750**. I then clicked on the packet, Follow > TCP stream. I then opened the file and found the flag **THM{http_raw_3xfiltr4ti0n_succ3ss}**.

ICMP is a network-layer protocol used for diagnostics and control (e.g., ping, TTL exceeded). Because it is commonly allowed through firewalls and typically inspected less strictly than TCP/UDP, attackers sometimes abuse ICMP to tunnel and exfiltrate data. Malicious actors encode data into ICMP payloads (echo request/reply, timestamp, info) and send it to a remote listener under their control.

Step 1: What is the flag found in the exfiltrated data through ICMP?

icmp.type == 8 and frame.len > 100							
No.	Time	Source	Destination	Protocol	Length	Info	
19	52.900000	192.168.1.101	10.0.0.254	ICMP	148	Echo (ping) request	id=0x0000, seq=0/0
1470	346079.900000	192.168.1.102	10.0.0.254	ICMP	148	Echo (ping) request	id=0x0000, seq=0/0
1709	432332.900000	192.168.1.101	10.0.0.254	ICMP	148	Echo (ping) request	id=0x0000, seq=0/0
3602	951013.900000	192.168.1.101	10.0.0.254	ICMP	148	Echo (ping) request	id=0x0000, seq=0/0
8392	2506101.900000	192.168.1.103	10.0.0.254	ICMP	148	Echo (ping) request	id=0x0000, seq=0/0

0000	45 00 00 94 00 01 00 00	40 01 ad 5b c0 a8 01 67	E @ . . [. . . g
0010	0a 00 00 fe 08 00 0d 63	00 00 00 00 3a 0d 0a 54	 c : . . . T
0020	48 4d 7b 31 63 6d 70 5f	33 63 68 30 5f 33 78 66	HM{1cmp_3ch0_3xf
0030	31 6c 74 72 34 74 31 30	6e 5f 73 75 63 63 33 73	1ltr4t10 n_succ3s
0040	73 7d 0d 0a 2d 2d 2d 0d	0a 46 69 6c 65 20 48 61	s} File Ha
0050	73 68 65 73 3a 0d 0a 20	20 73 65 63 72 65 74 2e	shes: . . . secret.
0060	74 78 74 3a 20 39 63 38	66 33 65 32 62 37 61 31	txt: 9c8 f3e2b7a1
0070	64 34 65 35 66 36 63 37	62 38 61 39 64 30 65 31	d4e5f6c7 b8a9d0e1
0080	66 32 61 33 62 0d 0a 20	20 62 61 63 6b 75 70 2e	f2a3b . . . backup.
0090	74 61 72 2e		tar.

I applied the filter **icmp.type == 8 and frame.len > 100** and Identified a very few number of packets so I was able to look through each of them and found the flag -

THM{1cmp_3ch0_3xf1ltr4t10n_succ3ss}.

Conclusion Summary

In this investigation, I successfully identified and analyzed multiple **data exfiltration techniques** across DNS, FTP, HTTP, and ICMP protocols using packet analysis and log-based investigation methods. By applying targeted filters, protocol-specific indicators, and traffic volume analysis, I was able to distinguish malicious activity from normal network behavior and trace exfiltration back to compromised internal hosts.

The DNS analysis revealed covert tunneling activity through suspicious subdomain patterns linked to *tunnelcorp.net*, with a single internal host generating a high volume of encoded queries. FTP analysis exposed unauthorized file transfers, including sensitive customer data, through both guest and root accounts, demonstrating how legacy protocols can be abused for large data theft. HTTP investigation highlighted data exfiltration hidden within POST requests to external domains, while ICMP analysis confirmed payload-based tunneling in echo request traffic.

Across all scenarios, I accurately identified **compromised internal IPs, exfiltrated files, and hidden indicators**, reinforcing practical skills in Wireshark analysis, traffic correlation, and attacker behavior detection. This exercise strengthened my ability to detect covert exfiltration channels, apply structured investigation workflows, and support real-world **SOC monitoring and incident response operations**.